

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Ульяновский государственный технический университет»
Кафедра «Вычислительная техника»

Операционные системы

(название дисциплины)

Лабораторная работа №4

Проектирование пула потоков

(название работы)

Выполнил

Студент группы ИВТИИБД-21

Мокшин Д.И.

(Фамилия И.О.)

Проверил(а):

ст. преподаватель кафедры «ВТ»

(должность)

Беляев К.С.

(Фамилия И.О.)

Ульяновск
2026

СОДЕРЖАНИЕ

1. Задание на лабораторную работу.....	3
2. Описание реализации.....	3
3. Тестирование.....	6
На рисунке 3.1 представлен результат работы программы.....	6
Рис. 3.1. Результат работы программы.....	6
4. Вывод.....	6
СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ.....	6

1. Задание на лабораторную работу

Требуется спроектировать и реализовать на языке C программный модуль «Пул потоков», базирующийся на библиотеке Pthreads. Система должна обеспечивать синхронизацию доступа к общей очереди задач с помощью мьютексов и семафоров (или условных переменных). Архитектура должна поддерживать асинхронную передачу задач от клиентского кода к свободным рабочим потокам. Необходимо предусмотреть механизм блокирующего ожидания при пустой очереди и процедуру корректной деинициализации (graceful shutdown), гарантирующую завершение всех активных процессов после выполнения поставленных задач.

2. Описание реализации

Реализация пула потоков основывается на структуре `task_node` Рис2.1, которая служит базовым элементом очереди. Каждый такой узел объединяет в себе указатель на целевую функцию, данные для выполнения (например, параметры для сложения чисел) и адрес следующего элемента, формируя связный список задач.

```
typedef struct task_node
{
    void (*function)(void *p);
    void *data;
    struct task_node *next;
} task;

task *queue_head = NULL;
task *queue_tail = NULL;
```

Рис 2.1 структура `task_node`

Процесс управления очередью возложен на функции `enqueue()` Рис2.2 и `dequeue()` Рис2.3. Первая отвечает за формирование нового узла и его вставку в конец списка, при этом она инкрементирует счетчик задач и сигнализирует семафору `sem_post` о появлении работы. Вторая функция выполняет

обратную операцию: извлекает данные из заголовка очереди для передачи потоку-исполнителю и корректно освобождает занимаемую узлом память.

```
int enqueue(task t)
{
    task *new_task = (task*)malloc(sizeof(task));
    if (new_task == NULL)
        return 1;

    new_task->function = t.function;
    new_task->data = t.data;
    new_task->next = NULL;

    pthread_mutex_lock(&queue_mutex);

    if (queue_tail == NULL)
    {
        queue_head = new_task;
        queue_tail = new_task;
    }
    else
    {
        queue_tail->next = new_task;
        queue_tail = new_task;
    }
    queue_count++;

    pthread_mutex_unlock(&queue_mutex);

    sem_post(&queue_sem);

    return 0;
}
```

Рис 2.2 функция enqueue

```
task dequeue()
{
    pthread_mutex_lock(&queue_mutex);

    task *temp = queue_head;
    task work;
    work.function = temp->function;
    work.data = temp->data;

    queue_head = queue_head->next;
    if (queue_head == NULL)
        queue_tail = NULL;

    queue_count--;

    pthread_mutex_unlock(&queue_mutex);

    free(temp);
    return work;
}
```

Рис 2.3 функция dequeue

Запуск системы осуществляется через функцию pool_init() Рис2.4, которая подготавливает семафор и порождает группу рабочих потоков. Каждый поток при входе в функцию worker() переходит в состояние ожидания на семафоре. Как только поступает сигнал о наличии задачи, поток проверяет флаги состояния: если работа пула продолжается или в очереди

еще остались элементы, он вызывает `dequeue()` и передает задачу в функцию `execute()` для непосредственного вычисления.

```
void pool_init(void)
{
    sem_init(&queue_sem, 0, 0);

    for (int i = 0; i < NUMBER_OF_THREADS; i++)
    {
        pthread_create(&workers[i], NULL, worker, NULL);
    }
}
```

Рис 2.4 функция `pool_init`

Интерфейс для пользователя представлен функцией `pool_submit()` Рис2.5, которая инкапсулирует создание структуры задачи и её постановку в очередь, возвращая статус выполнения операции. Завершение работы контролируется функцией `pool_shutdown()`, которая деактивирует флаг активности пула и поочередно ожидает остановки всех потоков через механизм `pthread_join()`. Финальным этапом является полная очистка системных ресурсов, включая мьютекс и семафор.

```
int pool_submit(void (*somefunction)(void *p), void *p)
{
    task newtask;
    newtask.function = somefunction;
    newtask.data = p;

    return enqueue(newtask);
}
```

Рис 2.5 `pool_submit`

Практическое применение реализовано в клиентском приложении, где подготавливаются наборы данных с числами для суммирования. После активации пула задачи распределяются между потоками, а по завершении расчетов инициируется корректная остановка всей системы.

3. Тестирование

На рисунке 3.1 представлен результат работы программы.

```
Инициализация пула потоков...
Отправка 5 задач в пул...
Задача 1: 0 + 0
Задача 2: 10 + 5
Задача 3: 20 + 10
Задача 4: 30 + 15
Задача 5: 40 + 20
Ожидание выполнения задач...
Вычисляю 0 + 0 = 0
Вычисляю 10 + 5 = 15
Вычисляю 20 + 10 = 30
Вычисляю 30 + 15 = 45
Вычисляю 40 + 20 = 60
Завершение работы пула...
Программа завершена
danil@WIN-CUBCPHMT5SL:~/lab4$
```

Рис. 3.1. Результат работы программы

4. Вывод

В ходе выполнения лабораторной работы разработан и протестирован пул потоков на языке C с использованием POSIX Threads. Реализованы функции инициализации, отправки задач, остановки пула. Для синхронизации доступа к очереди применён мьютекс, для уведомления потоков — семафор. Очередь реализована в виде связанного списка, что позволяет динамически добавлять любое количество задач.

СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

1. Керниган Б. У. Язык программирования С / Б. У. Керниган, Д. М. Ричи.
- 2-е изд. - М. : Вильямс, 2021. - 304 с.
2. Робачевский А. М. Операционная система UNIX / А. М. Робачевский. -
2-е изд. - СПб. : БХВ-Петербург, 2010. - 656 с.